C PROGRAMS

Syntax Check, Logical Review, Sample Outputs and Tables

12 uploaded programs reviewed

This document contains a program-wise review, representative sample input/output, and the complete source code supplied in the uploads. Compilation was checked with GCC using C11 mode.

Important: Successful compilation does not guarantee that a program is logically correct for every possible input. The notes below distinguish compilation status from algorithmic limitations.

Overall findings

Category	Finding
Files reviewed	12
C compilation	All 12 compile when the extensionless FIRST source is explicitly compiled as C
Programs needing attention	dag.c, stack(1).c, syntax.c, first
Programs with stated scope/input limitations	heap(1).c, keywords(1).c, predictive(1).c, recursive(1).c, recursive2(1).c, slr1(1).c, slr2(1).c
No functional issue for demonstrated purpose	test.c

Compilation note: the file named **first** has no .c extension. GCC may treat it as a linker input unless invoked with `-x c`. This is a filename/compiler invocation issue, not a C syntax error.

Program-wise status table

Source file	Purpose	Review
dag.c	DAG construction and Three Address Code generation	Whitespace / validation issues
heap(1).c	Heap memory allocation: First Fit, Best Fit and Worst Fit	Correct within fixed-block model
keywords(1).c	C keyword and identifier checking	Limited keyword list
predictive(1).c	Predictive parsing using a parsing table	Correct for stated grammar
recursive(1).c	Recursive descent parser for arithmetic expressions	Correct for stated grammar
recursive2(1).c	Recursive descent parser for the grammar S → (L) | a	Correct for stated grammar
slr1(1).c	SLR parser for arithmetic expressions	Correct for stated grammar
slr2(1).c	SLR parser for S → AB, A → a | ε, B → b | ε	Correct for fixed ab$ input
stack(1).c	Dynamic stack implementation	Full-stack behaviour nonstandard
static.c	Global and static variables	Correct for demo
syntax.c	Syntax tree and Three Address Code generation	Whitespace / validation issues
first	FIRST set computation	FIRST logic incomplete

1. dag.c

DAG construction and Three Address Code generation

Check result

Compiles successfully. DAG sharing works for repeated subexpressions in the supported expression format.

Issues / limitations

Spaces between operands and operators can stop parsing early because term() and expr() do not skip spaces before checking operators. Parentheses are advanced over without verifying that a closing ')' exists. There are no complete malformed-input or buffer-length checks.

Practical note

For the sample output below, enter the expression without spaces: x=a+b*c+b*c. The repeated subexpression b*c is represented by one DAG node and one temporary.

Sample input

```
x=a+b*c+b*c
```

Expected / observed sample output

```
Enter expression:

Three Address Code:
t1 = b * c
t2 = a + t1
t3 = t2 + t1
x = t3
```

Complete uploaded source code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

struct node
{
    char op, name[20], temp[20];
    struct node *l, *r;
};

struct node *nodes[100];
char s[100];
int p = 0, count = 0, countTemp = 0;

struct node *create(char op, char *name, struct node *l, struct node *r)
{
    for (int i = 0; i < count; i++)
    {
        if (nodes[i]->op == op)
        {
            if (!op && strcmp(nodes[i]->name, name) == 0)
                return nodes[i];

            if (op && nodes[i]->l == l && nodes[i]->r == r)
                return nodes[i];
        }
    }

    struct node *n = calloc(1, sizeof(struct node));
    n->op = op;
    if (name)
        strcpy(n->name, name);
    n->l = l;
    n->r = r;
    nodes[count++] = n;
    return n;
}

struct node *expr();

struct node *factor()
{
    while (s[p] == ' ')
        p++;

    if (s[p] == '-')
    {
        p++;
        return create('~', NULL, factor(), NULL);
    }
```

```
    if (s[p] == '(')
    {
        p++;
        struct node *n = expr();
        p++;
        return n;
    }

    char a[20];
    int i = 0;
    while (isalnum(s[p]) || s[p] == '_')
        a[i++] = s[p++];
    a[i] = '\0';

    return create(0, a, NULL, NULL);
}

struct node *term()
{
    struct node *n = factor();
    while (s[p] == '*' || s[p] == '/')
    {
        char op = s[p++];
        n = create(op, NULL, n, factor());
    }
    return n;
}

struct node *expr()
{
    struct node *n = term();
    while (s[p] == '+' || s[p] == '-')
    {
        char op = s[p++];
        n = create(op, NULL, n, term());
    }
    return n;
}

char *generate(struct node *n)
{
    if (!n->op)
        return n->name;
    if (n->temp[0])
        return n->temp;

    char a[20], b[20], t[20];
    strcpy(a, generate(n->l));

    if (n->op == '~')
        sprintf(t, "t%d", ++countTemp),
            printf("%s = -%s\n", t, a);
    else
    {
        strcpy(b, generate(n->r));
        sprintf(t, "t%d", ++countTemp);
        printf("%s = %s %c %s\n", t, a, n->op, b);
    }

    strcpy(n->temp, t);
    return n->temp;
}

int main()
{
    char lhs[20];
    printf("Enter expression: ");
    fgets(s, sizeof(s), stdin);

    char *eq = strchr(s, '=');
    if (!eq)
        return printf("Invalid expression\n"), 0;

    strncpy(lhs, s, eq - s);
    lhs[eq - s] = '\0';
    p = eq - s + 1;

    struct node *root = expr();

    printf("\nThree Address Code:\n");
    printf("%s = %s\n", lhs, generate(root));
    return 0;
}
```

2. heap(1).c

Heap memory allocation: First Fit, Best Fit and Worst Fit

Check result

Compiles successfully. The allocation-selection logic implements First Fit, Best Fit and Worst Fit for fixed-size memory blocks.

Issues / limitations

malloc() and scanf() return values are not checked. The program assumes a valid positive block count, positive sizes, process count and choice (1–3). Invalid choices simply result in no allocation. It marks an entire block allocated; it does not split a block or track internal fragmentation.

Practical note

The sample uses Best Fit. Each block can be allocated to only one process.

Sample input

```
3
100
500
200
2
3
100
400
150
```

Expected / observed sample output

```
Enter total number of memory blocks: Enter block size: Enter block size: Enter block size:
1.First Fit
2.Best Fit
3.Worst Fit
Enter choice: Enter number of processes:
Enter process size: Process 1 allocated to block of size 100

Enter process size: Process 2 allocated to block of size 500

Enter process size: Process 3 allocated to block of size 200

Final Heap Memory Status:
Block 1: Size = 100, Status = Allocated
Block 2: Size = 500, Status = Allocated
Block 3: Size = 200, Status = Allocated
```

Complete uploaded source code

```
#include <stdio.h>
#include <stdlib.h>

struct Block
{
    int size, free;
    struct Block *next;
};

int main()
{
    int n, i, choice, p, found;
    struct Block *head = NULL, *temp, *prev, *best;

    printf("Enter total number of memory blocks: ");
    scanf("%d", &n);

    // Create heap blocks dynamically
    for (i = 0; i < n; i++)
    {
        temp = (struct Block *)malloc(sizeof(struct Block));

        printf("Enter block size: ");
        scanf("%d", &temp->size);

        temp->free = 1;
        temp->next = NULL;

        if (head == NULL)
            head = temp;
        else
            prev->next = temp;

        prev = temp;
```

```
    }

    printf("\n1.First Fit\n2.Best Fit\n3.Worst Fit");
    printf("\nEnter choice: ");
    scanf("%d", &choice);

    printf("Enter number of processes: ");
    scanf("%d", &p);

    for (i = 0; i < p; i++)
    {
        int size;
        printf("\nEnter process size: ");
        scanf("%d", &size);

        temp = head;
        best = NULL;
        found = 0;

        while (temp != NULL)
        {
            if (temp->free && temp->size >= size)
            {

                if (choice == 1)
                {
                    best = temp;
                    break;
                }

                if (choice == 2 &&
                    (best == NULL || temp->size < best->size))
                    best = temp;

                if (choice == 3 &&
                    (best == NULL || temp->size > best->size))
                    best = temp;
            }
            temp = temp->next;
        }

        if (best != NULL)
        {
            best->free = 0;
            found = 1;
            printf("Process %d allocated to block of size %d\n",
                   i + 1, best->size);
        }

        if (!found)
            printf("Process %d cannot be allocated\n", i + 1);
    }

    printf("\nFinal Heap Memory Status:\n");

    temp = head;
    i = 1;

    while (temp != NULL)
    {
        printf("Block %d: Size = %d, Status = %s\n",
               i++, temp->size,
               temp->free ? "Free" : "Allocated");

        temp = temp->next;
    }

    // Release dynamically allocated heap blocks
    while (head != NULL)
    {
        temp = head;
        head = head->next;
        free(temp);
    }

    return 0;
}
```

3. keywords(1).c

C keyword and identifier checking

Check result

Compiles successfully. The listed words are detected and other strings are checked against the basic C identifier character rules.

Issues / limitations

The keyword list contains only 12 control-flow keywords, not all C keywords. Therefore a word such as int is reported as a valid identifier even though it is a C keyword. The program reads one whitespace-delimited word.

Practical note

Use this program as a flow-control-keyword and identifier checker, not as a complete C lexical analyser.

Sample input

```
while
```

Expected / observed sample output

```
Enter a word: while is a flow control keyword
```

Complete uploaded source code

```
#include <stdio.h>

int main()
{
    char str[50];
    const char *keywords[] = {
        "if", "else", "while", "do", "switch",
        "case", "default", "continue", "return",
        "goto", "for", "break"};
    int n = sizeof(keywords) / sizeof(keywords[0]);
    int i, j, found = 0;
    printf("Enter a word: ");
    scanf("%49s", str);
    for (i = 0; i < n; i++)
    {
        j = 0;
        while (str[j] == keywords[i][j] && str[j] != '\0')
            j++;
        if (str[j] == '\0' && keywords[i][j] == '\0')
        {
            found = 1;
            break;
        }
    }

    if (found)
    {
        printf("%s is a flow control keyword\n", str);
    }
    else
    {
        if ((str[0] >= 'A' && str[0] <= 'Z') ||
            (str[0] >= 'a' && str[0] <= 'z') ||
            (str[0] == '_'))
        {

            found = 1;

            for (i = 1; str[i] != '\0'; i++)
            {
                if (!((str[i] >= 'A' && str[i] <= 'Z') ||
                      (str[i] >= 'a' && str[i] <= 'z') ||
                      (str[i] >= '0' && str[i] <= '9') ||
                      (str[i] == '_')))
                {
                    found = 0;
                    break;
                }
            }

            if (found)
                printf("\n%s is a valid identifier", str);
            else
                printf("\n%s is not a valid identifier", str);
        }
        else
        {
            printf("\n%s is not a valid identifier", str);
        }
    }
```

```
    return 0;
}
```

4. predictive(1).c

Predictive parsing using a parsing table

Check result

Compiles successfully. The parsing table and stack logic are consistent with the expression grammar E→TQ, Q→+TQ|ε, T→FR, R→*FR|ε, F→i|(E).

Issues / limitations

The input uses i as the terminal representing id, and must end with $. The firstfollow() function is defined but never called. Stack capacity is not checked.

Practical note

Use i+i*i$ as a valid sample. The program expects i, not the two-character token id.

Sample input

```
i+i*i$
```

Expected / observed sample output

```
Enter input ending with $:
Valid string
```

Complete uploaded source code

```
#include <stdio.h>

char stack[100];
char input[100];
int top = -1, ip = 0;

void push(char c)
{
    stack[++top] = c;
}

char pop()
{
    return stack[top--];
}

char peek()
{
    return stack[top];
}

void firstfollow()
{
    printf("First(E) = { (, id }\n");
    printf("First(Q) = { +, e }\n");
    printf("First(T) = { (, id }\n");
    printf("First(R) = { *, e }\n");
    printf("First(F) = { (, id }\n");
}

int col(char ch)
{
    switch (ch)
    {
    case 'i':
        return 0;
    case '+':
        return 1;
    case '*':
        return 2;
    case '(':
        return 3;
    case ')':
        return 4;
    case '$':
        return 5;
    }
    return -1;
}

int row(char ch)
{
    switch (ch)
    {
    case 'E':
        return 0;
    case 'Q':
        return 1;
    case 'T':
```

```
        return 2;
    case 'R':
        return 3;
    case 'F':
        return 4;
    }
    return -1;
}

char *table[5][6] =
    {
        {"TQ", "", "", "TQ", "", ""},
        {"", "+TQ", "", "", "#", "#"},
        {"FR", "", "", "FR", "", ""},
        {"", "#", "*FR", "", "#", "#"},
        {"i", "", "", "(E)", "", ""}};

int main()
{
    int j;

    printf("Enter input ending with $: ");
    scanf("%99s", input);

    push('$');
    push('E');

    while (peek() != '$')
    {
        char x = peek();
        char a = input[ip];

        if (x == a)
        {
            pop();
            ip++;
        }
        else if (x >= 'A' && x <= 'Z')
        {
            int r = row(x);
            int c = col(a);

            if (r == -1 || c == -1 || table[r][c][0] == '\0')
            {
                printf("Invalid string\n");
                return 0;
            }

            pop();

            if (table[r][c][0] != '#')
            {
                int len = 0;

                while (table[r][c][len] != '\0')
                    len++;

                for (j = len - 1; j >= 0; j--)
                    push(table[r][c][j]);
            }
        }
        else
        {
            printf("\nInvalid string\n");
            return 0;
        }
    }

    if (input[ip] == '$')
        printf("\nValid string");
    else
        printf("\nInvalid string");

    return 0;
}
```

5. recursive(1).c

Recursive descent parser for arithmetic expressions

Check result

Compiles successfully. The parser implements the usual left-recursion-free expression grammar with +, *, parentheses and alphanumeric identifiers.

Issues / limitations

The grammar does not include subtraction or division. Identifiers must begin with a letter; later characters may be letters or digits. scanf reads one token, so spaces are not accepted.

Practical note

a+b*c is accepted; a+*b is rejected.

Sample input

```
a+b*c
```

Expected / observed sample output

```
Enter a string: String accepted
```

Complete uploaded source code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

char ip[100];
int pos = 0;

void E();
void Eprime();
void T();
void Tprime();
void F();

void error()
{
    printf("Syntax error at position %d\n", pos);
    exit(1);
}

void E()
{
    T();
    Eprime();
}

void Eprime()
{
    if (ip[pos] == '+')
    {
        pos++;
        T();
        Eprime();
    }
}

void T()
{
    F();
    Tprime();
}

void Tprime()
{
    if (ip[pos] == '*')
    {
        pos++;
        F();
        Tprime();
    }
}

void F()
{
    if (ip[pos] == '(')
    {
        pos++;
        E();

        if (ip[pos] == ')')
```

```
            pos++;
        else
            error();
    }
    else if (isalpha((unsigned char)ip[pos]))
    {
        pos++;

        while (isalnum((unsigned char)ip[pos]))
            pos++;
    }
    else
    {
        error();
    }
}

int main()
{
    printf("Enter a string: ");
    scanf("%99s", ip);

    E();

    if (ip[pos] == '\0')
        printf("String accepted\n");
    else
        error();

    return 0;
}
```

6. recursive2(1).c

Recursive descent parser for the grammar S → (L) | a

Check result

Compiles successfully. The parser implements S→(L)|a, L→SL′, L′→,SL′|ε.

Issues / limitations

The only simple terminal is a. The program reads a whitespace-delimited string and uses exit(0) for syntax errors, which can misleadingly signal successful process termination to the operating system.

Practical note

(a,a) is accepted; (a,) and a,a are rejected.

Sample input

```
(a,a)
```

Expected / observed sample output

```
Enter string: String is accepted
```

Complete uploaded source code

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

char ip[100];
int pos = 0;

void S();
void L();
void Lprime();

void error()
{
    printf("Syntax error and string not accepted!\n");
    exit(0);
}

void S()
{
    if (ip[pos] == '(')
    {
        pos++;
        L();

        if (ip[pos] == ')')
            pos++;
        else
            error();
    }
    else if (ip[pos] == 'a')
    {
        pos++;
    }
    else
    {
        error();
    }
}

void L()
{
    S();
    Lprime();
}

void Lprime()
{
    if (ip[pos] == ',')
    {
        pos++;
        S();
        Lprime();
    }
}

int main()
{
    printf("Enter string: ");
    scanf("%99s", ip);

    S();
```

```
    if (ip[pos] == '\0')
        printf("String is accepted\n");
    else
        error();

    return 0;
}
```

7. slr1(1).c

SLR parser for arithmetic expressions

Check result

Compiles successfully. The ACTION and GOTO tables and reduction cases implement the standard expression grammar E→E+T|T, T→T*F|F, F→(E)|id.

Issues / limitations

Tokens are entered separately: id, +, *, (,). The program stores only the first character of each token in its display-symbol stack, so id appears as i in the printed stack. gotocol() is unused. Stack bounds are not explicitly checked.

Practical note

For the sample, enter 5 tokens: id, +, id, *, id.

Sample input

```
5
id
+
id
*
id
```

Expected / observed sample output

```
Enter number of tokens: Enter tokens (id, +, *, (, )):

Stack               Input           Action
$0                id              Shift 5
$0 i5                +               Reduce F -> id
$0 F3                +               Reduce T -> F
$0 T2                +               Reduce E -> T
$0 E1                +               Shift 6
$0 E1 +6                id              Shift 5
$0 E1 +6 i5                *               Reduce F -> id
$0 E1 +6 F3                *               Reduce T -> F
$0 E1 +6 T9                *               Shift 7
$0 E1 +6 T9 *7                id              Shift 5
$0 E1 +6 T9 *7 i5                $               Reduce F -> id
$0 E1 +6 T9 *7 F10                $                Reduce T -> T*F
$0 E1 +6 T9                $               Reduce E -> E+T
$0 E1                $               String Accepted
```

Complete uploaded source code

```
#include <stdio.h>
#include <string.h>

#define MAX 100

int action[12][6] = {
    {5, 0, 0, 4, 0, 0},
    {0, 6, 0, 0, 0, 999},
    {0, -2, 7, 0, -2, -2},
    {0, -4, -4, 0, -4, -4},
    {5, 0, 0, 4, 0, 0},
    {0, -6, -6, 0, -6, -6},
    {5, 0, 0, 4, 0, 0},
    {5, 0, 0, 4, 0, 0},
    {0, 6, 0, 0, 11, 0},
    {0, -1, 7, 0, -1, -1},
    {0, -3, -3, 0, -3, -3},
    {0, -5, -5, 0, -5, -5}
};

int gotot[12][3] = {
    {1, 2, 3},
    {-1, -1, -1},
    {-1, -1, -1},
    {-1, -1, -1},
    {8, 2, 3},
    {-1, -1, -1},
    {-1, 9, 3},
    {-1, -1, 10},
    {-1, -1, -1},
    {-1, -1, -1},
    {-1, -1, -1},
    {-1, -1, -1}
};

int getcol(char *c)
{
```

```
    if (strcmp(c, "id") == 0)
        return 0;
    if (strcmp(c, "+") == 0)
        return 1;
    if (strcmp(c, "*") == 0)
        return 2;
    if (strcmp(c, "(") == 0)
        return 3;
    if (strcmp(c, ")") == 0)
        return 4;
    if (strcmp(c, "$") == 0)
        return 5;

    return -1;
}

int gotocol(char c)
{
    if (c == 'E')
        return 0;
    if (c == 'T')
        return 1;
    if (c == 'F')
        return 2;

    return -1;
}

void print(int st[], char symbol[], int top)
{
    int i;

    for (i = 0; i <= top; i++)
        printf("%c%d ", symbol[i], st[i]);
}

void printprod(int r)
{
    switch (r)
    {
    case 1:
        printf("Reduce E -> E+T");
        break;

    case 2:
        printf("Reduce E -> T");
        break;

    case 3:
        printf("Reduce T -> T*F");
        break;

    case 4:
        printf("Reduce T -> F");
        break;

    case 5:
        printf("Reduce F -> (E)");
        break;

    case 6:
        printf("Reduce F -> id");
        break;
    }
}

int main()
{
    char input[MAX][10];
    char symbol[MAX];
    int st[MAX];
    int n, i;
    int top = 0;

    printf("Enter number of tokens: ");
    scanf("%d", &n);

    if (n < 1 || n >= MAX)
    {
        printf("Invalid number of tokens!\n");
        return 1;
    }

    printf("Enter tokens (id, +, *, (, )):\n");

    for (i = 0; i < n; i++)
        scanf("%9s", input[i]);

    strcpy(input[n], "$");

    st[0] = 0;
    symbol[0] = '$';

    i = 0;

    printf("\n%-20s %-15s %s\n",
           "Stack", "Input", "Action");
```

```
while (1)
{
    int state = st[top];
    int col = getcol(input[i]);
    int act;

    if (col == -1)
    {
        printf("Invalid input\n");
        break;
    }

    act = action[state][col];

    print(st, symbol, top);
    printf("%-15s %-15s ", "", input[i]);

    if (act == 0)
    {
        printf("String Rejected\n");
        break;
    }

    if (act == 999)
    {
        printf("String Accepted\n");
        break;
    }

    if (act > 0)
    {
        printf("Shift %d\n", act);

        top++;
        st[top] = act;
        symbol[top] = input[i][0];

        i++;
    }
    else
    {
        int r = -act;
        int lhs;
        int rhslen;
        int pre;

        printprod(r);
        printf("\n");

        if (r == 1)
        {
            lhs = 0;
            rhslen = 3;
        }
        else if (r == 2)
        {
            lhs = 0;
            rhslen = 1;
        }
        else if (r == 3)
        {
            lhs = 1;
            rhslen = 3;
        }
        else if (r == 4)
        {
            lhs = 1;
            rhslen = 1;
        }
        else if (r == 5)
        {
            lhs = 2;
            rhslen = 3;
        }
        else
        {
            lhs = 2;
            rhslen = 1;
        }

        top = top - rhslen;

        pre = st[top];

        top++;
        st[top] = gotot[pre][lhs];

        if (lhs == 0)
            symbol[top] = 'E';
        else if (lhs == 1)
            symbol[top] = 'T';
        else
            symbol[top] = 'F';
    }
}

return 0;
```

```
}
```

8. slr2(1).c

SLR parser for S → AB, A → a | ε, B → b | ε

Check result

Compiles successfully. The table is designed for S→AB, A→a|ε, B→b|ε.

Issues / limitations

The input is hard-coded as ab$; the user cannot enter another string. The table can be exercised against other inputs only after changing the source. The sample demonstrates the hard-coded ab input.

Practical note

The intended grammar can generate ε, a, b and ab; this particular program only runs the ab case.

Sample input

```
Fixed in source: ab$
```

Expected / observed sample output

```
Input: ab$

Stack                Input           Action
0                  ab$             Shift 3
03                  b$              Reduce A -> a
02                  b$              Shift 5
025                  $               Reduce B -> b
024                  $               Reduce S -> AB
01                  $               String accepted
```

Complete uploaded source code

```
#include <stdio.h>
#include <string.h>

#define MAX 100

int action[6][3] = {
    {3, -3, -3},
    {0, 0, 999},
    {0, 5, -5},
    {0, -2, -2},
    {0, 0, -1},
    {0, 0, -4}};

int gotot[6][3] = {
    {1, 2, 0},
    {0, 0, 0},
    {0, 0, 4},
    {0, 0, 0},
    {0, 0, 0},
    {0, 0, 0}};

int getcol(char c)
{
    if (c == 'a')
        return 0;
    if (c == 'b')
        return 1;
    if (c == '$')
        return 2;

    return -1;
}

void print(int st[], int top)
{
    int i;

    for (i = 0; i <= top; i++)
        printf("%d", st[i]);
}

void printprod(int r)
{
    switch (r)
    {
    case 1:
        printf("Reduce S -> AB");
        break;

    case 2:
        printf("Reduce A -> a");
        break;

    case 3:
```

```
        printf("Reduce A -> epsilon");
        break;

    case 4:
        printf("Reduce B -> b");
        break;

    case 5:
        printf("Reduce B -> epsilon");
        break;
    }
}

int main()
{
    char in[MAX] = "ab$";
    int st[MAX];
    int top = 0;
    int ip = 0;

    st[0] = 0;

    printf("Input: %s\n\n", in);
    printf("%-20s %-15s %s\n",
           "Stack", "Input", "Action");

    while (1)
    {
        int state;
        int col;
        int act;

        state = st[top];
        col = getcol(in[ip]);

        if (col == -1)
        {
            printf("Invalid input\n");
            return 0;
        }

        act = action[state][col];

        print(st, top);
        printf("%-15s %-15s ",
               "", in + ip);

        if (act == 0)
        {
            printf("Error\n");
            break;
        }

        if (act == 999)
        {
            printf("String accepted\n");
            break;
        }

        if (act > 0)
        {
            printf("Shift %d\n", act);

            top++;
            st[top] = act;
            ip++;
        }
        else
        {
            int r = -act;
            int lhs;
            int rhslen;
            int pre;

            printprod(r);
            printf("\n");

            if (r == 1)
            {
                lhs = 0;
                rhslen = 2;
            }
            else if (r == 2)
            {
                lhs = 1;
                rhslen = 1;
            }
            else if (r == 3)
            {
                lhs = 1;
                rhslen = 0;
            }
            else if (r == 4)
            {
                lhs = 2;
                rhslen = 1;
            }
```

```
            else
            {
                lhs = 2;
                rhslen = 0;
            }

            top = top - rhslen;
            pre = st[top];

            top++;
            st[top] = gotot[pre][lhs];
        }
    }

    return 0;
}
```

9. stack(1).c

Dynamic stack implementation

Check result

Compiles successfully. Dynamic allocation, push, pop, display and exit operations work for ordinary inputs.

Issues / limitations

When the stack is full, the program removes the current top element and then pushes the new value. This is not standard stack overflow handling; it silently changes the stack. A conventional implementation should print 'Stack is full' and leave the stack unchanged. scanf() results are not checked.

Practical note

The sample demonstrates normal push, display, pop and exit operations without filling the stack.

Sample input

```
3
1
10
1
20
3
2
3
4
```

Expected / observed sample output

```
Enter the stack size:
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: Enter the element: 10 pushed into the stack.

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: Enter the element: 20 pushed into the stack.

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: Stack elements are:
20
10

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: Popped element: 20

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: Stack elements are:
10

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: Memory released. Exiting...
```

Complete uploaded source code

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *stack = NULL;
    int size, top = -1;
    int choice, value;

    printf("Enter the stack size: ");
```

```
scanf("%d", &size);

if (size <= 0)
{
    printf("Invalid stack size!\n");
    return 1;
}

while (1)
{
    printf("\n1. Push\n");
    printf("2. Pop\n");
    printf("3. Display\n");
    printf("4. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
    case 1:
        printf("Enter the element: ");
        scanf("%d", &value);

        if (top == size - 1)
        {
            printf("Stack is full!\n");
            printf("Removing top element: %d\n",
                   stack[top]);

            // Remove the current top element
            top--;

            if (top == -1)
            {
                free(stack);
                stack = NULL;
            }
            else
            {
                int *temp = realloc(
                    stack, (top + 1)*sizeof(int));

                if (temp != NULL)
                {
                    stack = temp;
                }
            }
        }

        // Allocate memory for the new element
        int *temp = realloc(
            stack, (top + 2) * sizeof(int));

        if (temp == NULL)
        {
            printf("Memory allocation failed!\n");
            break;
        }

        stack = temp;
        stack[++top] = value;

        printf("%d pushed into the stack.\n",
               value);
        break;

    case 2:
        if (top == -1)
        {
            printf("Stack is empty!\n");
        }
        else
        {
            printf("Popped element: %d\n",
                   stack[top]);
            top--;

            if (top == -1)
            {
                free(stack);
                stack = NULL;
            }
            else
            {
                int *temp = realloc(
                    stack, (top + 1) * sizeof(int));

                if (temp != NULL)
                {
                    stack = temp;
                }
            }
        }
        break;

    case 3:
        if (top == -1)
```

```
            {
                printf("Stack is empty!\n");
            }
            else
            {
                printf("Stack elements are:\n");
                for (int i = top; i >= 0; i--)
                {
                    printf("%d\n", stack[i]);
                }
            }
            break;

        case 4:
            free(stack);
            printf("Memory released. Exiting...\n");
            return 0;

        default:
            printf("Invalid choice!\n");
        }
    }

    return 0;
}
```

10. static.c

Global and static variables

Check result

Compiles successfully. The global variable x, file-scope static variable y and local static variable z are initialized and printed correctly.

Issues / limitations

No functional issue for the demonstrated purpose. The static keyword on y gives it file scope (internal linkage); z retains its value between calls to main's containing function, though main is called only once here.

Practical note

Expected output is 10 20 30.

Sample input

```
No input required.
```

Expected / observed sample output

```
10 20 30
```

Complete uploaded source code

```
#include <stdio.h>

int x = 10;
static int y = 20;

int main()
{
    static int z = 30;

    printf("%d %d %d\n", x, y, z);

    return 0;
}
```

11. syntax.c

Syntax tree and Three Address Code generation

Check result

Compiles successfully. It builds an expression tree and generates three-address code for supported arithmetic expressions.

Issues / limitations

Like dag.c, it does not skip spaces before checking operators in term() and expr(), so spaced expressions can be truncated. It advances past ')' without validating it and does not validate the complete expression. Unlike dag.c, it does not perform common-subexpression sharing.

Practical note

Enter x=a+b*c to obtain the sample output. This program creates a syntax tree, not a shared DAG.

Sample input

```
x=a+b*c
```

Expected / observed sample output

```
Enter expression:

Three Address Code:
t1 = b * c
t2 = a + t1
x = t2
```

Complete uploaded source code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

struct node
{
    char op, name[20], temp[20];
    struct node *l, *r;
};

char s[100];
int p = 0, count = 0;

struct node *create(char op, char *name, struct node *l, struct node *r)
{
    struct node *n = calloc(1, sizeof(struct node));
    n->op = op;
    if (name)
        strcpy(n->name, name);
    n->l = l;
    n->r = r;
    return n;
}

struct node *expr();

struct node *factor()
{
    while (s[p] == ' ')
        p++;

    if (s[p] == '-')
    {
        p++;
        return create('~', NULL, factor(), NULL);
    }

    if (s[p] == '(')
    {
        p++;
        struct node *n = expr();
        p++;
        return n;
    }

    char a[20];
    int i = 0;
    while (isalnum(s[p]) || s[p] == '_')
        a[i++] = s[p++];
    a[i] = '\0';

    return create(0, a, NULL, NULL);
```

```
}

struct node *term()
{
    struct node *n = factor();
    while (s[p] == '*' || s[p] == '/')
    {
        char op = s[p++];
        n = create(op, NULL, n, factor());
    }
    return n;
}

struct node *expr()
{
    struct node *n = term();
    while (s[p] == '+' || s[p] == '-')
    {
        char op = s[p++];
        n = create(op, NULL, n, term());
    }
    return n;
}

char *generate(struct node *n)
{
    if (!n->op)
        return n->name;

    char a[20], b[20], t[20];
    strcpy(a, generate(n->l));

    if (n->op == '~')
        sprintf(t, "t%d", ++count),
            printf("%s = -%s\n", t, a);
    else
    {
        strcpy(b, generate(n->r));
        sprintf(t, "t%d", ++count);
        printf("%s = %s %c %s\n", t, a, n->op, b);
    }

    strcpy(n->temp, t);
    return n->temp;
}

int main()
{
    char lhs[20];
    printf("Enter expression: ");
    fgets(s, sizeof(s), stdin);

    char *eq = strchr(s, '=');
    if (!eq)
        return printf("Invalid expression\n"), 0;

    strncpy(lhs, s, eq - s);
    lhs[eq - s] = '\0';
    p = eq - s + 1;

    struct node *root = expr();

    printf("\nThree Address Code:\n");
    printf("%s = %s\n", lhs, generate(root));
    return 0;
}
```

12. first

FIRST set computation

Check result

Compiles as C when the compiler is told to treat the extensionless file as C source (for example, gcc -x c first -o first).

Issues / limitations

The algorithm does not compute FIRST sets for general context-free grammars. It looks only at the first RHS symbol, does not propagate through nullable symbols, and does not correctly handle epsilon or sequences. It also assumes uppercase single-character non-terminals and has fixed-size arrays without bounds checks.

Practical note

The sample E→a, E→b, E→c works because every production begins with a terminal. Do not rely on this implementation for general FIRST-set questions.

Sample input

```
3
E->a
E->b
E->c
```

Expected / observed sample output

```
Enter number of productions: Enter productions (Example: E->E+T):

FIRST sets:
FIRST(E) = { a b c }
```

Complete uploaded source code

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

char lhs[20];
char rhs[20][20][10];
int n, count[20];

char first[26][20][10];
int fcount[26];

int add(int nt, char *s)
{
    for (int i = 0; i < fcount[nt]; i++)
        if (strcmp(first[nt][i], s) == 0)
            return 0;

    strcpy(first[nt][fcount[nt]++], s);
    return 1;
}

int main()
{
    printf("Enter number of productions: ");
    scanf("%d", &n);

    printf("Enter productions (Example: E->E+T):\n");

    for (int i = 0; i < n; i++)
    {
        char line[50];
        scanf("%s", line);

        int nt = line[0] - 'A';
        int j = 3, k = 0;

        lhs[i] = line[0];

        while (line[j])
        {
            if (isupper(line[j]))
            {
                rhs[i][k][0] = line[j++];
                rhs[i][k++][1] = '\0';
            }
            else if (islower(line[j]))
            {
                int p = 0;
                while (islower(line[j]))
                    rhs[i][k][p++] = line[j++];
                rhs[i][k++][p] = '\0';
            }
```

```
            else
            {
                rhs[i][k][0] = line[j++];
                rhs[i][k++][1] = '\0';
            }
        }

        count[i] = k;
    }

    int changed;

    do
    {
        changed = 0;

        for (int i = 0; i < n; i++)
        {
            int nt = lhs[i] - 'A';
            char *s = rhs[i][0];

            if (isupper(s[0]))
            {
                int x = s[0] - 'A';

                for (int j = 0; j < fcount[x]; j++)
                    changed |= add(nt, first[x][j]);
            }
            else
            {
                changed |= add(nt, s);
            }
        }

    } while (changed);

    printf("\nFIRST sets:\n");

    for (int i = 0; i < 26; i++)
    {
        if (fcount[i])
        {
            printf("FIRST(%c) = { ", i + 'A');

            for (int j = 0; j < fcount[i]; j++)
                printf("%s ", first[i][j]);

            printf("}\n");
        }
    }

    return 0;
}
```